Tenable Nessus: Vulnerability Scan & Remediation

Oct 21, 2026

Username and stored secrets redacted in screenshots.

Objective

I installed Tenable Nessus Essentials on my laptop, scanned it, and remediated the Medium findings it reported. The scan found 7 Medium-severity instances; after replacing Splunk's default TLS certificate with a CA-signed one, 1 remains.

The brief asked for four things: install Nessus, run a Basic Network Scan using "Scan for all web vulnerabilities (quick)", remediate at least one Medium-or-higher finding on my own machine, then rescan to confirm.

Environment

One Windows laptop acted as both the scanner and the target.

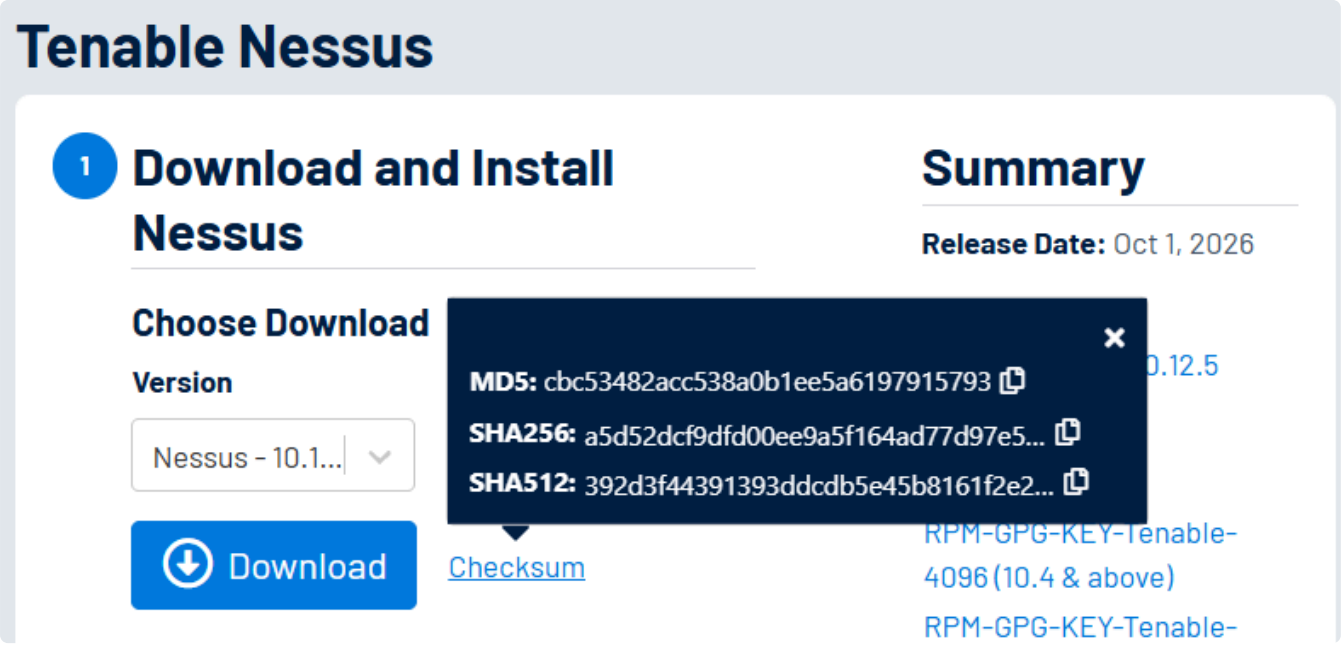
Item	Detail
Host	Acer Nitro AN515-43 (hostname host DNS host.lan)
OS	Windows 11 Home 25H2
Scanner	Nessus Essentials 10.12.5
Licence	5 hosts (1 used); plugins on a 30-day delay
Target	192.168.x.x (Wi-Fi address)
Scan type	Basic Network Scan, unauthenticated
Services on the target	Splunk Enterprise 10.6 (8089 management, 5432 internal PostgreSQL), Nessus web UI (8834)

Two limits shape the results. The scan ran without credentials, so it only saw network-facing services, not installed software. The Essentials plugin feed runs 30 days behind the paid feed, so vulnerabilities published in the last month would not be detected.

Installation and configuration

The installer was verified before running, and the only setup problem was antivirus slowing the plugin compile.

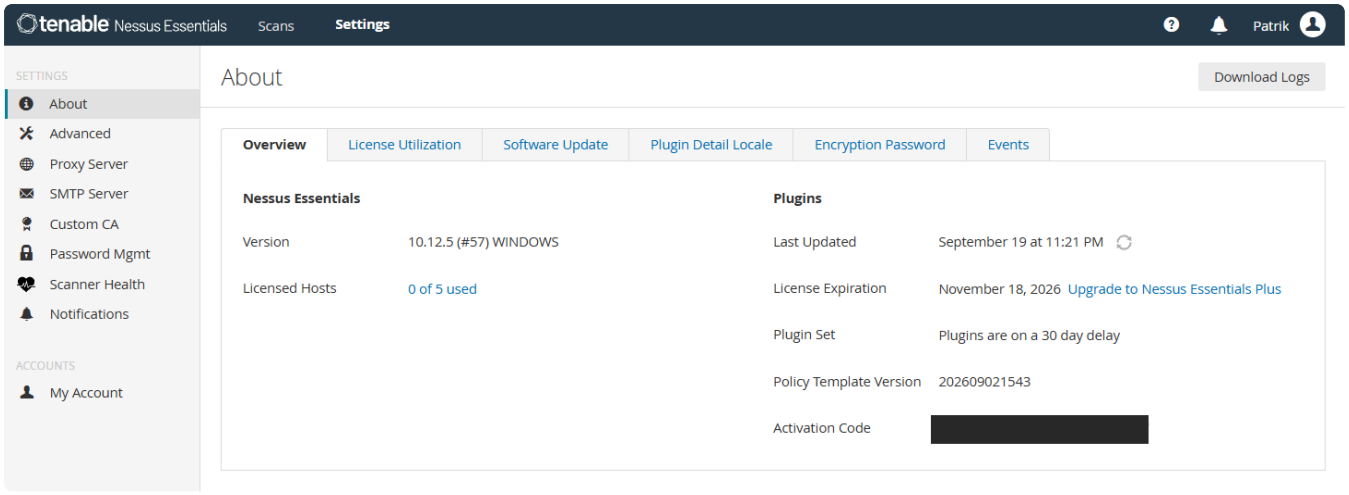
1. **Verified the installer.** I compared the SHA256 checksum on Tenable's download page with the downloaded file. PowerShell's `(Get-FileHash .\Nessus-10.12.5-x64.msi -Algorithm SHA256).Hash -eq '<Tenable's value>'` returned `True`.
2. **Registered Nessus Essentials** at https://localhost:8834, as the brief required.
3. **Fixed a slow plugin compile.** Nessus looked frozen while downloading plugins. Task Manager showed Microsoft Defender at 74% CPU, scanning every plugin file Nessus wrote. I added the folders and processes from Tenable's documented allowlist as Defender exclusions. Defender dropped to 8.8% CPU, and Nessus went from 3.4% to 52.9% CPU and from 2.2 to 6.1 MB/s disk.
4. **Updated plugins** via Settings → Software Update → Manual Software Update, then signed out and back in.



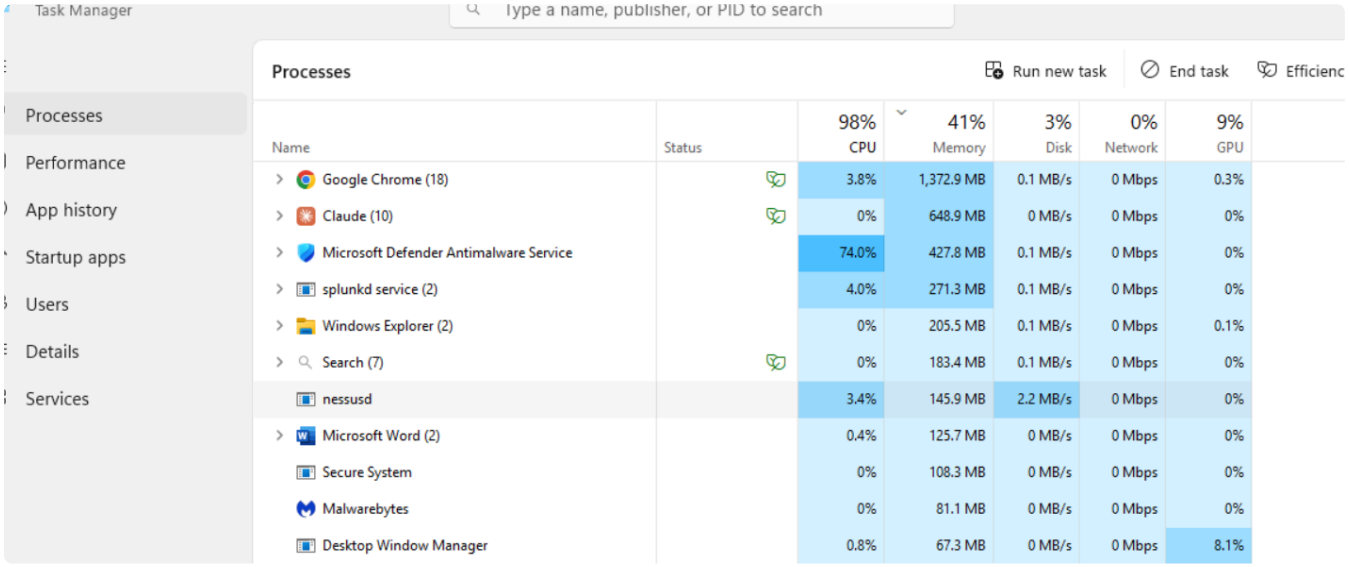
SHA256 checksum published on Tenable's download page

```
PS C:\Users\      \Downloads> (Get-FileHash .\Nessus-10.12.5-x64.msi -Algorithm SHA256).Hash -eq 'a5d52dcf9dfd00ee9a5f164
ad77d97e5d4d78e2bcd5d62e6de0d57af0f0b9745'
True
PS C:\Users\      \Downloads>
```

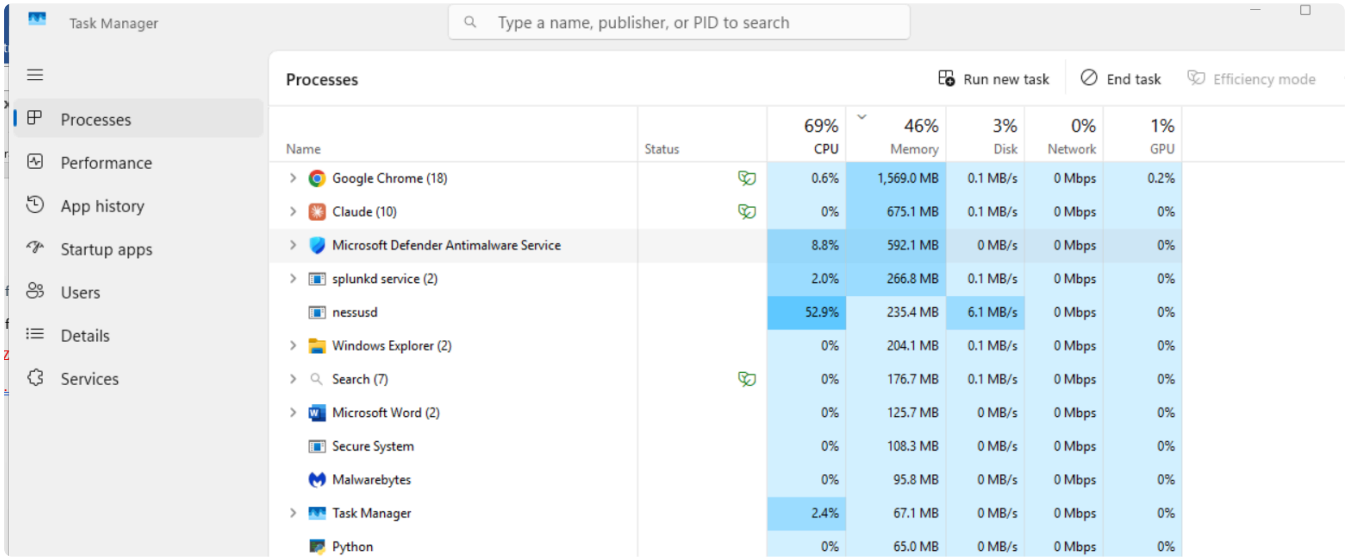
Get-FileHash comparison: the downloaded installer matches Tenable's checksum



Nessus Essentials licence



Before: Defender at 74% CPU, nessusd at 3.4%



After the allowlist exclusions: Defender at 8.8%, nessusd at 52.9%

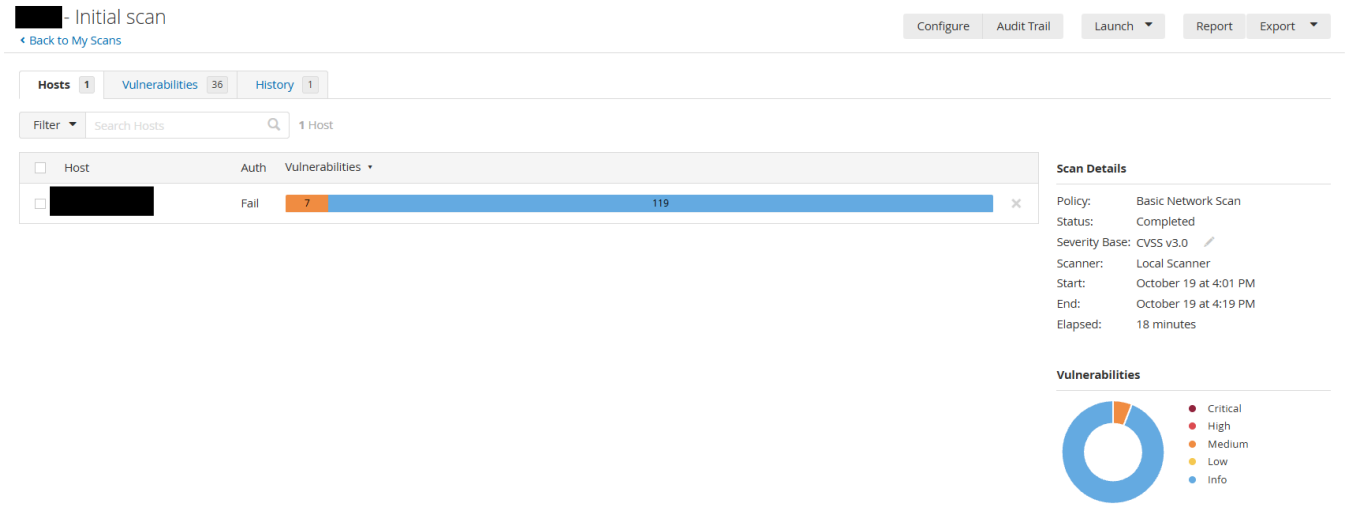
Initial scan and findings

The initial scan (19 Oct, 18 minutes) found 7 Medium and 119 Info findings. All three Medium findings were TLS certificate problems.

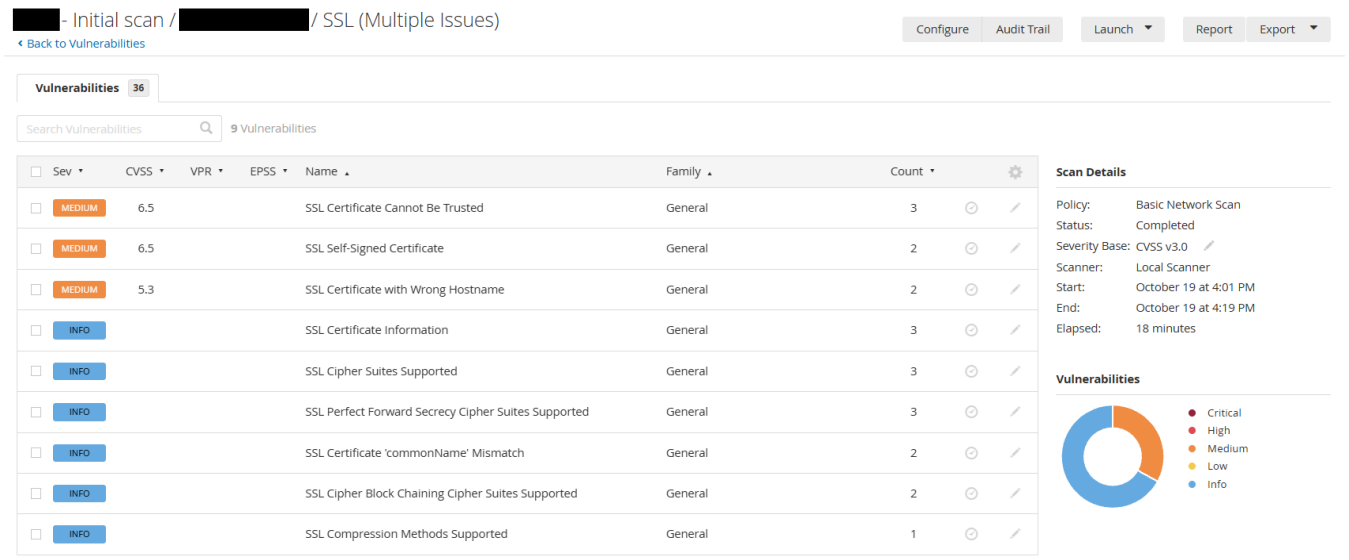
Finding	CVSS v3	Ports	What it means
SSL Certificate Cannot Be Trusted	6.5	8089, 5432, 8834	The certificate is not signed by an authority Nessus trusts
SSL Self-Signed Certificate	6.5	8089, 5432	The chain ends in a self-signed certificate (Splunk's default SplunkCommonCA)
SSL Certificate with Wrong Hostname	5.3	8089, 5432	The certificate's Common Name (SplunkServerDefaultCert) does not match the host ([redacted])

The Vulnerabilities tab showed 36, not 7. That number counts grouped rows, while the host bar counts each vulnerability per port. Three unique Medium issues across several ports add up to 7.

Ports 8089 and 5432 both belong to Splunk: its management port and its internal PostgreSQL database. Both presented the same default certificate, signed by SplunkCommonCA. Port 8834 is Nessus itself.



Initial scan: 7 Medium and 119 Info findings



Initial scan: three Medium certificate findings

Risk assessment and choosing a fix

The real-world risk was low, but I remediated the certificate on Splunk's ports anyway, because that is what Nessus's Solution asks for.

Exposure check. `netstat` showed splunkd listening on 0.0.0.0:8089, meaning every network interface. However, Windows Firewall was enabled on all profiles with the default inbound action (block), and no inbound rule allowed Splunk. Other devices on the network most likely could not reach this laptop port on 8089. The scan reached it only because Nessus ran on the same machine.

Options considered:

Option	Outcome	Decision
Add SplunkCommonCA to Nessus as trusted	Hides the finding. The same CA ships with every Splunk download, so trusting it trusts any Splunk server, including an attacker's.	Rejected: masks the problem
Bind Splunk to 127.0.0.1 only	Removes network exposure, but the certificate stays untrusted	Rejected: mitigation, not remediation

Option	Outcome	Decision
Replace the certificate with one from my own CA	Fixes trust, self-signing and hostname together	Chosen: matches the Solution and Splunk's documentation

Remediation steps

I built a private CA (Patrik CA), issued Splunk a certificate from it for host.lan, and told Nessus to trust that CA. I followed Splunk's guide "How to create and sign your own TLS certificates" on help.splunk.com, using the OpenSSL bundled with Splunk (`splunk cmd openssl`), with two corrections described below.

Part 1: the CA

1. Created the CA's private key, encrypted with a password.
2. Created the CA's certificate signing request (CSR), Common Name "Patrik CA".
3. Signed the CSR with the CA's own key to produce the root certificate, valid 3 years.

Correction 1: Splunk's documented signing command produced a certificate without `CA:TRUE`. The settings file has a header with a named section, `[v3_ca]`, but the command given in Splunk's documentation never names it, and OpenSSL only reads the default section, not a named one, unless told to. I caught this by inspecting the certificate with `openssl x509 -text`, then re-signed it with `-extensions v3_ca`. Without the fix, the CA could not have validated as a signer and the finding would have stayed. An alternative fix was removing the `[v3_ca]` header from the settings file.

Part 2: Splunk's certificate

1. Created a separate private key for Splunk.
2. Created a CSR with Common Name host.lan, the hostname Nessus recorded for the host.
3. Patrik CA signed it with `-extensions v3_srv`, valid 3 years.

Correction 2: Splunk's server signing command had the same problem: the settings sit under `[v3_srv]` but the command never names it. I added `-extensions v3_srv` so the certificate actually includes its settings.

Addition: Splunk's server settings file has no hostname entry, so I added `subjectAltName = DNS:`host.lan`, IP:`192.168.x.x for software that checks the SAN. Nessus's Wrong Hostname check uses the Common Name, which I set to host.lan in the server CSR.

Part 3: putting it into use

1. Combined the server certificate, its key and the CA certificate into one file, as Splunk's "How to prepare TLS certificates for use with the Splunk platform" requires.
2. Backed up `etc\system\local\server.conf`, then edited it and set `serverCert` to the path of the combined certificate file (myNewServerCertificate.pem) and `sslPassword` to the server key's password under `[sslConfig]`. Splunk encrypted the password on restart.
3. Restarted splunkd.
4. Pasted Patrik CA's certificate into Nessus → Settings → Custom CA.

Verification before rescanning. `openssl s_client` against 8089, and against 5432 with `-starttls postgres`, returned `subject=CN=host.lan` and `issuer=CN=Patrik CA` on both ports.

CA certificate check (`openssl x509 -text`, extract):

```
Issuer: CN=Patrik CA
Subject: CN=Patrik CA
X509v3 Basic Constraints: critical
    CA:TRUE
X509v3 Key Usage: critical
    Certificate Sign, CRL Sign
```

Server certificate check:

```
subject=CN=host.lan
X509v3 Subject Alternative Name:
    DNS:host.lan    , IP Address:192.168.x.x
```

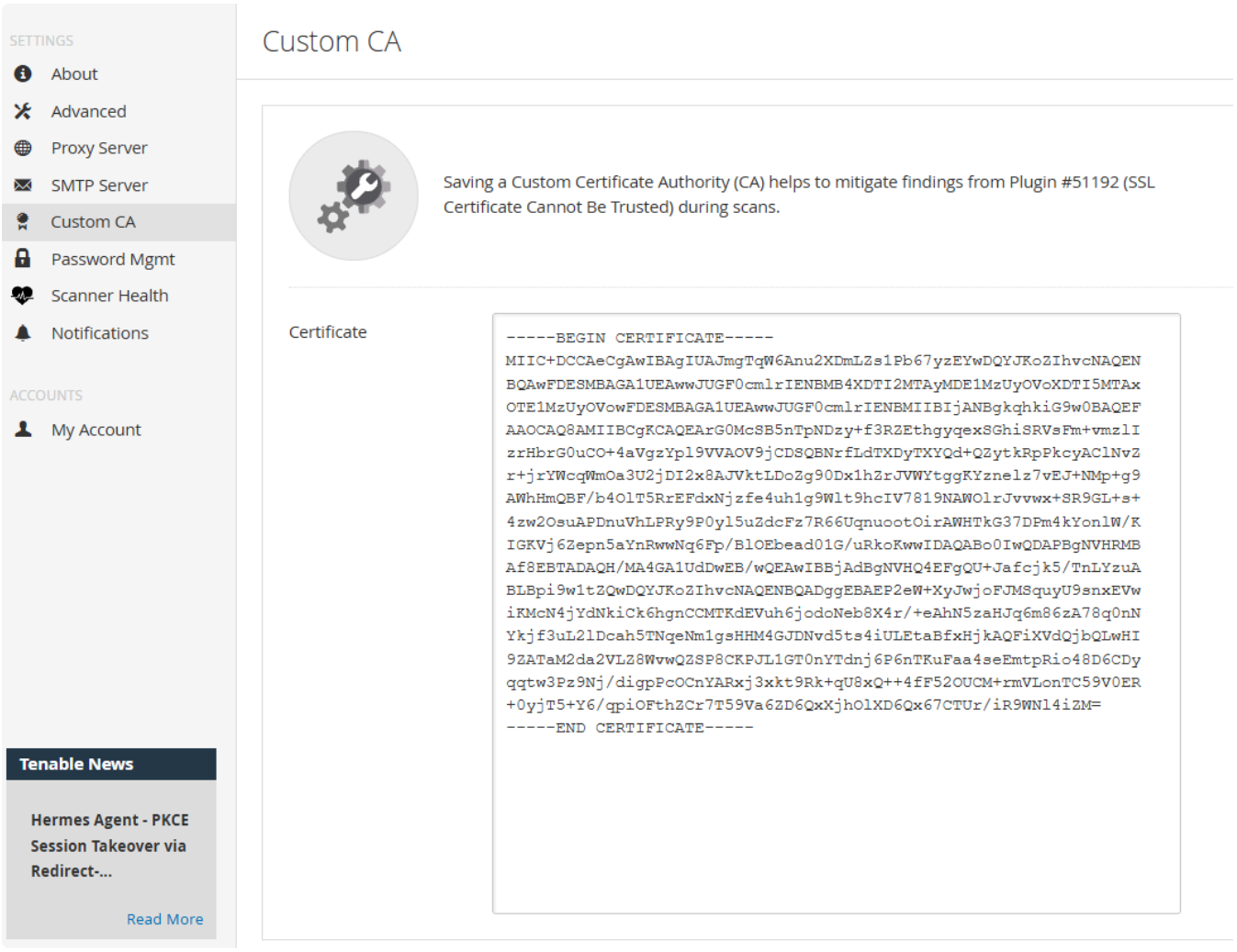
Certificate presented on 8089 and 5432 after the restart:

```
8089: subject=CN=host.lan  issuer=CN=Patrik CA
5432: subject=CN=host.lan  issuer=CN=Patrik CA
```

```
[general]
serverName = [redacted]
pass4SymmKey = $8$[redacted]

[sslConfig]
serverCert = $SPLUNK_HOME\etc\auth\mycerts\myNewServerCertificate.pem
sslPassword = $8$[redacted]
```

server.conf after the restart: serverCert points to the new certificate; sslPassword encrypted by Splunk



Patrik CA's public certificate saved in Nessus → Settings → Custom CA

Results

The rescan (21 Oct, 19 minutes) cut Medium instances from 7 to 1, and one configuration change fixed both Splunk ports.

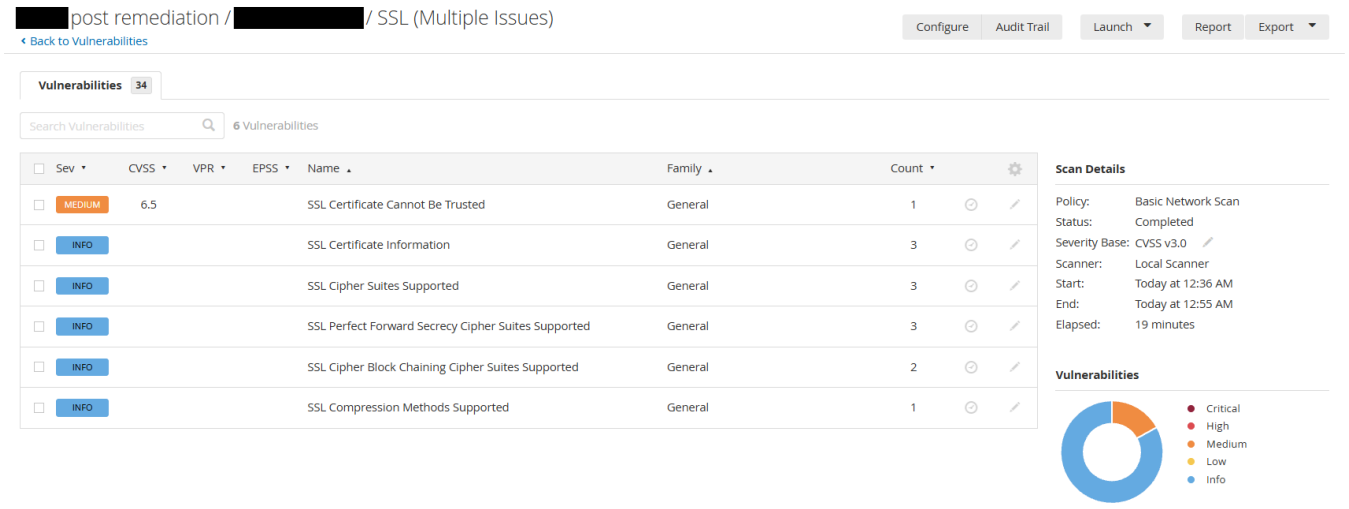
Finding	Before (ports)	After (ports)
SSL Certificate Cannot Be Trusted	8089, 5432, 8834	8834
SSL Self-Signed Certificate	8089, 5432	none
SSL Certificate with Wrong Hostname	8089, 5432	none
SSL Certificate 'commonName' Mismatch (Info)	8089, 5432	none
Medium instances	**7**	**1**
Total instances	126	115

Splunk's PostgreSQL on 5432 takes its certificate from the same `serverCert` setting as splunkd, so it was fixed without a separate change. The `-starttls postgres` check confirmed this directly.

The remaining finding is Nessus's own certificate on 8834, signed by "Nessus Certification Authority".



Rescan: 1 Medium and 114 Info findings



Rescan: only SSL Certificate Cannot Be Trusted remains, on 8834 (Nessus)

Lessons learned

- **Verify vendor commands by checking the output.** Splunk's documented CA command silently produced a certificate without `CA:TRUE`. Inspecting it with `openssl x509 -text` caught the error before it cost a failed rescan.
- **Remediation and mitigation are different.** Binding Splunk to localhost would have removed the finding without fixing the certificate. Trusting SplunkCommonCA would only have hidden it.

- **Check real exposure before rating risk.** A scan run from the same machine bypasses the firewall, so a reachable port in the scan is not proof of network exposure.
- **Read counts carefully.** Nessus's grouped Vulnerabilities count (36) and per-port instance count (7 Medium) measure different things.
- **Hostname checks use CN and SAN.** Nessus's Wrong Hostname plugin compares the certificate's Common Name with the host's names, so I set CN=`host.lan` and added the SAN for modern clients.
- **Know the scanner's limits.** This was an unauthenticated scan on a 30-day-delayed plugin feed, so it saw network services only, not missing patches or the newest vulnerabilities.
- **Security tools can collide.** Defender's real-time scanning slowed Nessus's plugin compile until I applied Tenable's documented allowlist.

Next steps (beyond this assignment scope)

- **Fix 8834 the same way:** issue a certificate for Nessus from Patrik CA (CA steps skipped, as it already exists) and install it using Tenable's documented procedure.
- **Run a credentialed scan** to see installed software and missing patches, which this unauthenticated scan could not.
- **Explore Tenable Vulnerability Management** (optional in the brief): deploy an agent and try other scan types.
- **Review the Defender exclusions** now that the plugin compile is finished, and keep or remove them deliberately.